EDGE INTELLIGENCE

LAB ASSESSMENT - 2

P. Meghana

25MML0010

TASK – 1

1. Data Jar

```
[4]: # DATA JAR
import numpy as np
import tensorflow as tf
from tensorflow import keras

(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

print("dtype:", x_train.dtype, "min,max:", x_train.min(), x_train.max())
print("shapes: x_train, y_train, x_test, y_test ->", x_train.shape, y_train.shape, x_test.shape, y_test.shape)
```

```
C:\Users\MEGHANA\anaconda3\Lib\site-packages\keras\src\export\tf2onnx_lib.py:8: FutureWarning: In the future `np.object` will be defined as the corresp
onding NumPy scalar.
  if not hasattr(np, "object"):
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 4s 0us/step
dtype: float32 min,max: 0.0 1.0
shapes: x_train, y_train, x_test, y_test -> (60000, 28, 28) (60000,) (10000, 28, 28) (10000,)
```

2. Task Jar

3. Model Jar

```
[5]: # TASK JAR
# The task is image classification on the MNIST dataset (classifying handwritten digits)

# MODEL JAR
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])
model.summary()
```

```
C:\Users\MEGHANA\anaconda3\Lib\site-packages\keras\src\layers\reshaping\flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim` argument t
o a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)

Trainable params: 101,770 (397.54 KB)

Non-trainable params: 0 (0.00 B)

4. Loss Jar

[6]:
```
# LOSS JAR
model.compile(
    optimizer='sgd',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

5. Learning Jar

[7]:
```
# LEARNING JAR
history = model.fit(
    x_train, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.1,
    verbose=2
)
```

```
Epoch 1/10
1688/1688 - 10s - 6ms/step - accuracy: 0.8247 - loss: 0.7005 - val_accuracy: 0.9167 - val_loss: 0.3248
Epoch 2/10
1688/1688 - 8s - 5ms/step - accuracy: 0.9024 - loss: 0.3513 - val_accuracy: 0.9317 - val_loss: 0.2603
Epoch 3/10
1688/1688 - 8s - 5ms/step - accuracy: 0.9158 - loss: 0.2999 - val_accuracy: 0.9370 - val_loss: 0.2317
Epoch 4/10
1688/1688 - 8s - 5ms/step - accuracy: 0.9245 - loss: 0.2695 - val_accuracy: 0.9422 - val_loss: 0.2106
Epoch 5/10
1688/1688 - 9s - 5ms/step - accuracy: 0.9311 - loss: 0.2466 - val_accuracy: 0.9482 - val_loss: 0.1933
Epoch 6/10
1688/1688 - 8s - 5ms/step - accuracy: 0.9362 - loss: 0.2285 - val_accuracy: 0.9513 - val_loss: 0.1804
Epoch 7/10
1688/1688 - 8s - 5ms/step - accuracy: 0.9396 - loss: 0.2130 - val_accuracy: 0.9533 - val_loss: 0.1713
Epoch 8/10
1688/1688 - 8s - 5ms/step - accuracy: 0.9438 - loss: 0.2001 - val_accuracy: 0.9592 - val_loss: 0.1599
Epoch 9/10
1688/1688 - 10s - 6ms/step - accuracy: 0.9469 - loss: 0.1885 - val_accuracy: 0.9598 - val_loss: 0.1520
Epoch 10/10
1688/1688 - 9s - 5ms/step - accuracy: 0.9499 - loss: 0.1781 - val_accuracy: 0.9620 - val_loss: 0.1457
```

6. Accuracy Jar

[8]:
```python
# ACCURACY JAR
loss, acc = model.evaluate(x_test, y_test, verbose=2)
print("Test loss:", loss, "Test accuracy:", acc)
```

```
313/313 - 1s - 4ms/step - accuracy: 0.9515 - loss: 0.1714
Test loss: 0.1713857501745224 Test accuracy: 0.9514999985694885
```

7. Saving the model using pickle

[10]:
```python
#Saving the model using pickle
import pickle
import os

pickle_file_name = 'my_keras_model.pkl'
with open(pickle_file_name, 'wb') as f:
    pickle.dump(model, f)
print(f"Model saved using pickle to '{pickle_file_name}'")
```

```
Model saved using pickle to 'my_keras_model.pkl'
```

8. Loading the model using pickle

[11]:
```python
#Loading the model using pickle
with open(pickle_file_name, 'rb') as f:
    loaded_pickled_model = pickle.load(f)
print(f"Model loaded successfully from '{pickle_file_name}' using pickle.")
loaded_pickled_model.summary()
```

```
Model loaded successfully from 'my_keras_model.pkl' using pickle.
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

Total params: 101,772 (397.55 KB)

Trainable params: 101,770 (397.54 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 2 (12.00 B)

9. Size of the model

[12]:
```python
#Size of the model
file_size_bytes = os.path.getsize(pickle_file_name)
file_size_kb = file_size_bytes / 1024
file_size_mb = file_size_bytes / (1024 * 1024)
print(f"Size of the saved pickled model file ('{pickle_file_name}'): {file_size_bytes} bytes ({file_size_kb:.2f} KB / {file_size_mb:.2f} MB)")
```

```
Size of the saved pickled model file ('my_keras_model.pkl'): 427704 bytes (417.68 KB / 0.41 MB)
```

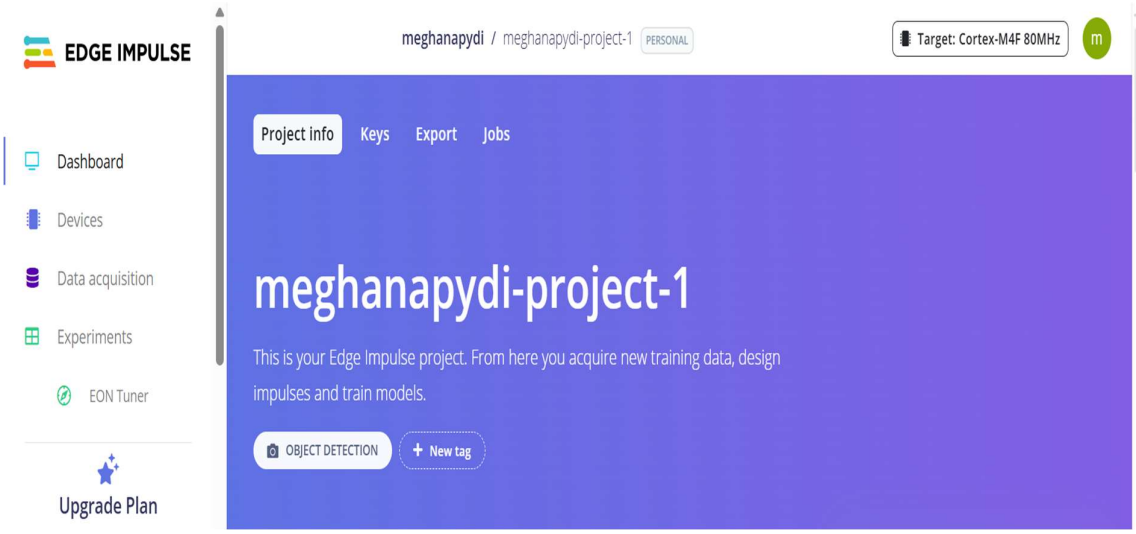
10. Evaluating the loaded pickled model

```
[13]: #Evaluating the loaded pickled model
if 'loaded_pickled_model' in locals() and 'x_test' in locals() and 'y_test' in locals():
    loss_loaded, acc_loaded = loaded_pickled_model.evaluate(x_test, y_test, verbose=2)
    print(f"Test loss (loaded pickled model): {loss_loaded:.4f}, Test accuracy (loaded pickled model): {acc_loaded:.4f}")
```

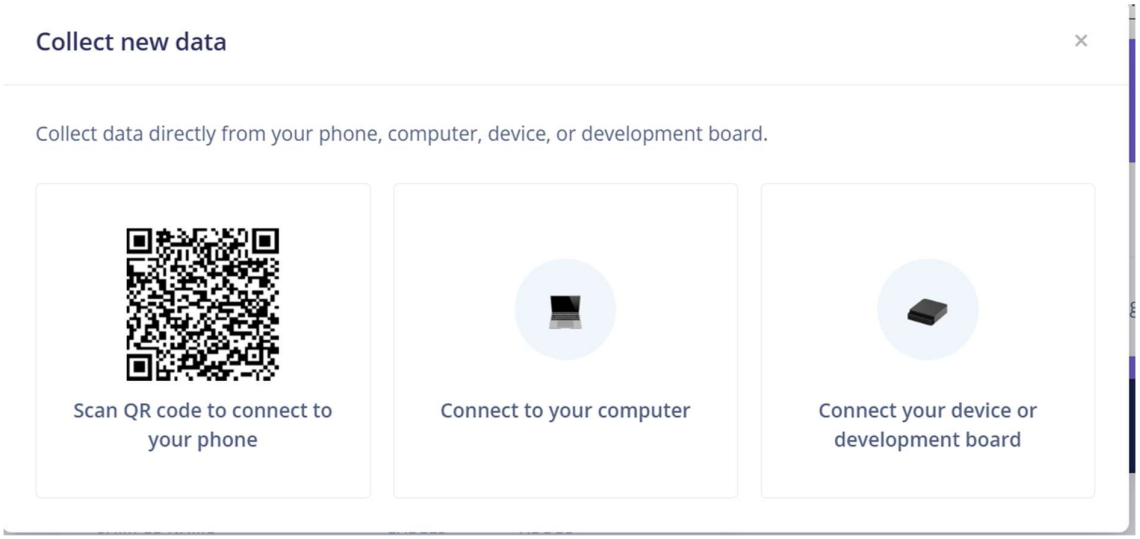
```
313/313 - 2s - 6ms/step - accuracy: 0.9515 - loss: 0.1714
Test loss (loaded pickled model): 0.1714, Test accuracy (loaded pickled model): 0.9515
```

TASK-2

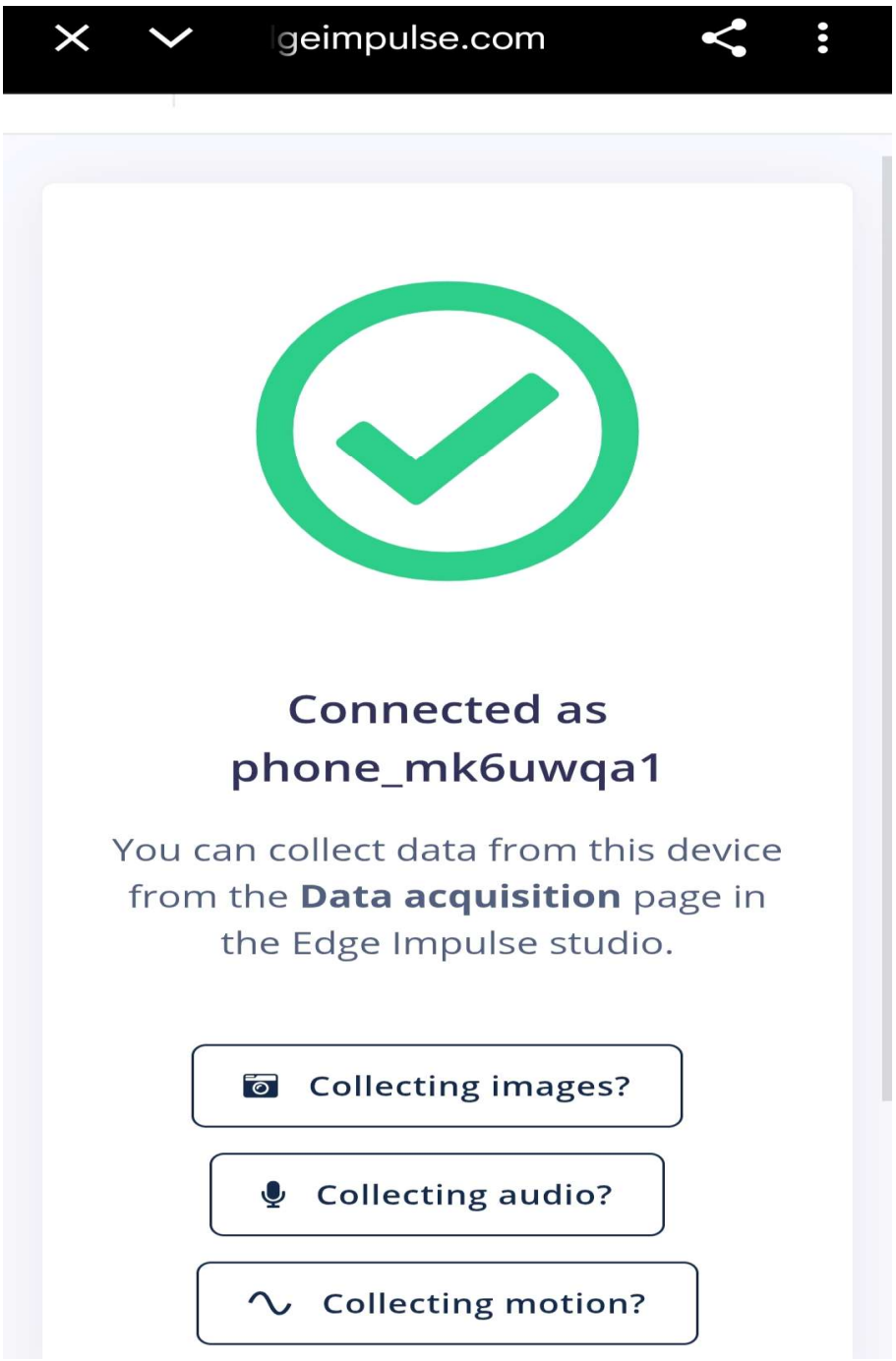
1. Logged in and Created a Project in Edge Impulse Framework



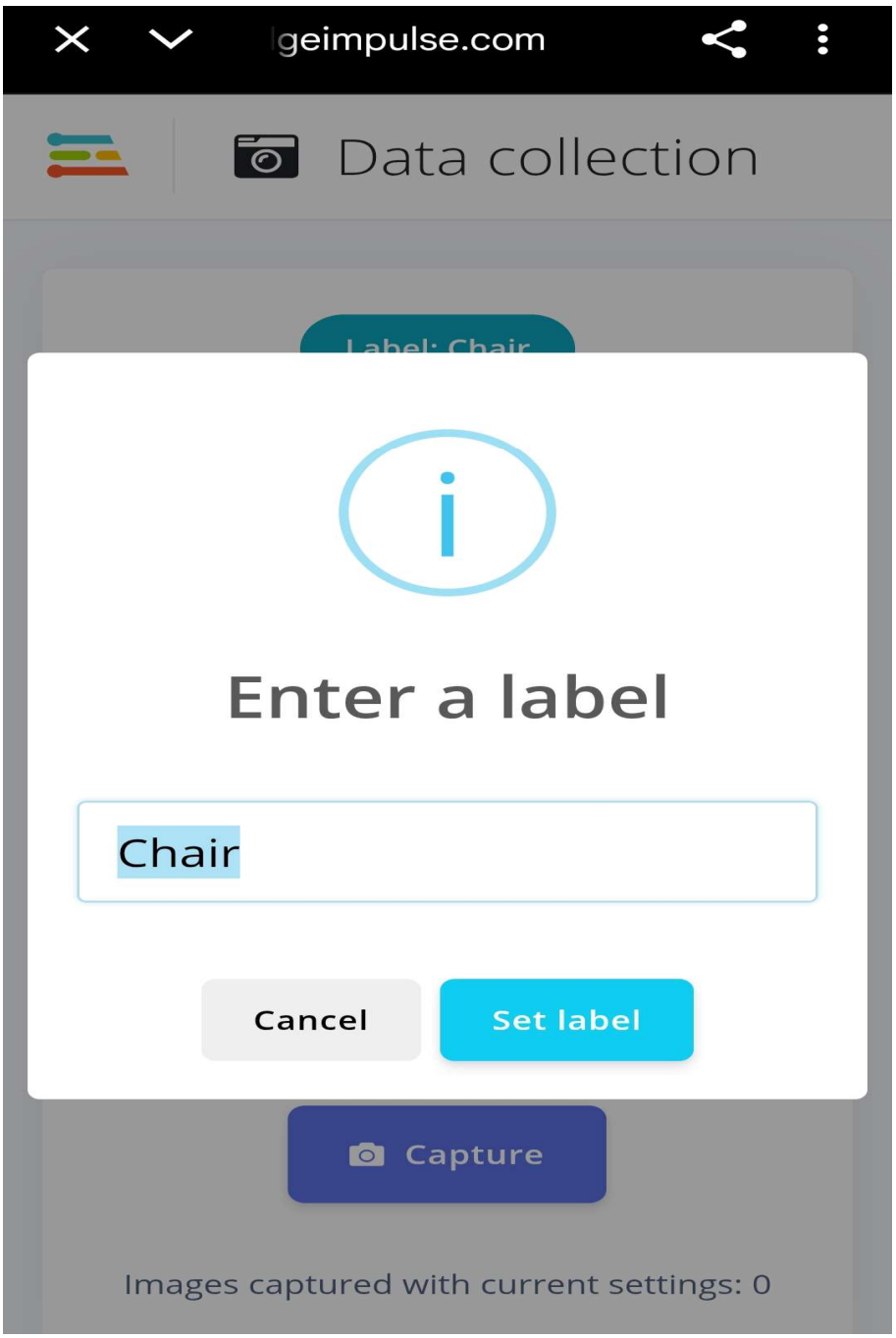
2. Collected New Data as part of Data Acquisition using phone



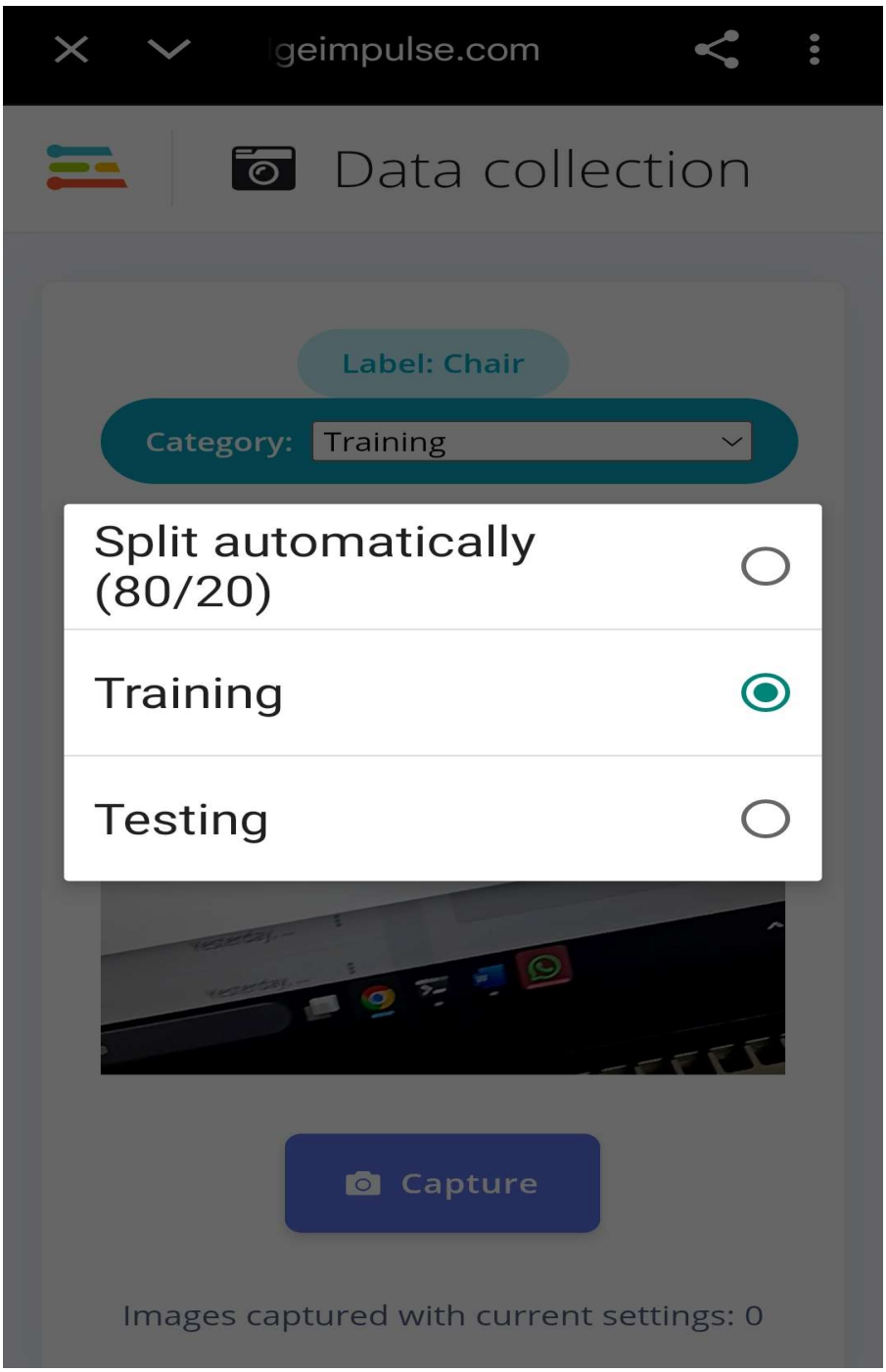
3. Connected phone using given QR code



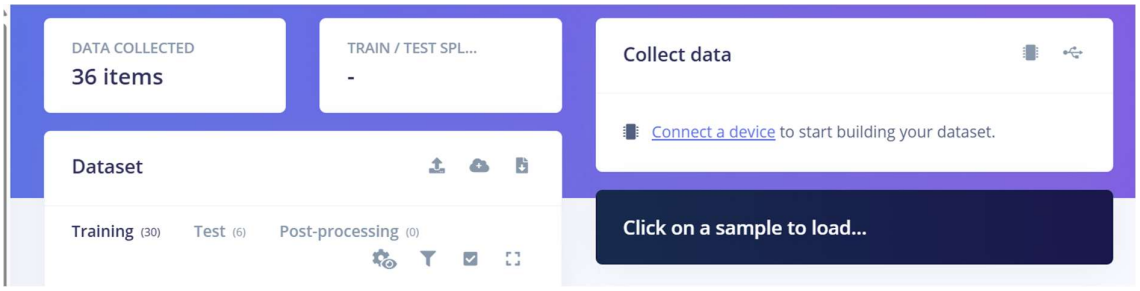
4. Entered relevant labels while collecting data

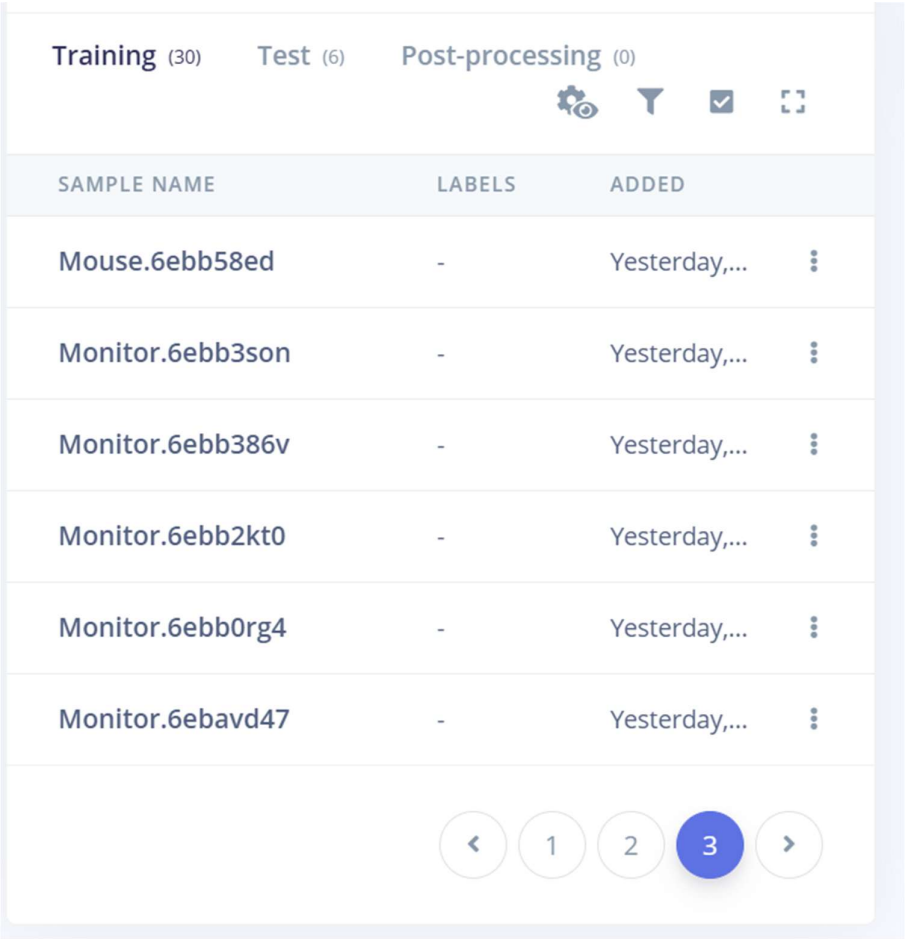


5. Collected data for both training and testing



6. Data uploaded to Edge Impulse Framework


DATA COLLECTED
36 items
TRAIN / TEST SPL...
-
Collect data
Connect a device to start building your dataset.
Dataset
Training (30)
Test (6)
Post-processing (0)
Click on a sample to load...


Training (30)
Test (6)
Post-processing (0)
SAMPLE NAME
LABELS
ADDED
Mouse.6ebb58ed
-
Yesterday,...
Monitor.6ebb3son
-
Yesterday,...
Monitor.6ebb386v
-
Yesterday,...
Monitor.6ebb2kt0
-
Yesterday,...
Monitor.6ebb0rg4
-
Yesterday,...
Monitor.6ebavd47
-
Yesterday,...
1
2
3